

1. 题目深度解构

题意重述

有两类商品：

1. **纪念品**：共有 n 个，价格为 a_i ，必须全部买。
2. **VIP 优惠券**：共有 m 张，第 j 张价格为 b_j ，参数为 c_j ，可以选择买或不买。

如果买了一张参数为 c 的优惠券，那么**下一次购买**的物品价格 x 会变成：

$$\max(x - c, 0)$$

这个“下一次购买”可以是纪念品，也可以是另一张 VIP 优惠券。

每个物品最多买一次，购买顺序任意。目标是**最小化**买完所有纪念品的总花费。

约束分析

$$1 \leq n, m \leq 5 \cdot 10^5$$

并且所有测试数据中 n, m 总和不超过：

$$5 \cdot 10^5$$

因此我们需要：

$$O((n + m) \log(n + m))$$

或者线性级别的算法。

不能做状态 DP，也不能枚举购买顺序。

题目类型判定

这是一道 **思维转化 + 区间覆盖 + 扫描线** 题。

表面上是购买顺序优化，实际上可以转化为：

用若干优惠券区间去覆盖若干纪念品价格区间，最大化覆盖长度。

2. 思维路径推导

从朴素想法开始

最直接的思路是 DP：

- 当前有哪些纪念品买了；
- 哪些优惠券买了；

- 当前是否有一张优惠券效果等待使用。

但显然不可能，因为 n, m 都可以达到 $5 \cdot 10^5$ 。

所以我们必须放弃“模拟购买顺序”，寻找购买行为背后的结构。

第一个观察：购买序列会分解成若干条链

因为纪念品不会产生新的优惠效果，所以一旦买了纪念品，这条优惠链就结束了。

例如一条购买链可能是：

优惠券 1 $\rightarrow$ 优惠券 2 $\rightarrow$ 优惠券 3 $\rightarrow$ 纪念品

每张优惠券只影响它后面的一个物品。

所以整个购买过程可以看成若干条链，每条链最终以一个纪念品结束。

没有必要买一张最后没有被使用的优惠券，因为它只会增加花费。

第二个观察：一张优惠券其实是一段区间

考虑一张优惠券：

(b, c)

它的价格是 b ，优惠参数是 c 。

如果它被用来优惠一个价格为 x 的物品，那么这一步的总效果是：

- 先花 b 买优惠券；
- 再让后面的物品少付 $\min(c, x)$ 。

所以它带来的净节省是：

$$\min(c, x) - b$$

如果 $c \leq b$ ，这张优惠券永远不会带来正收益，可以直接忽略。

如果 $c > b$ ，它可以理解成覆盖价格轴上的区间：

$[b, c]$

为什么？

因为它最多能把价格区间中从 b 到 c 的这一段“抵消掉”。

第三个观察：一个纪念品就是一段区间

一个价格为 a 的纪念品，如果完全原价购买，需要支付：

$$a$$

可以把它看成价格轴上的区间：

$$[0, a]$$

如果某些优惠券最终作用在这个纪念品所在的链上，那么这些优惠券对应的区间 $[b, c]$ 会覆盖 $[0, a]$ 的某些部分。

被覆盖的长度，就是可以省掉的钱。

于是，对于一个纪念品来说：

$$\text{实际花费} = a - \text{被优惠券区间覆盖的长度}$$

Aha! Moment：购买顺序变成了区间覆盖

原问题等价于：

- 有 n 条纪念品线段 $[0, a_i]$ ，以及 m 条优惠券线段 $[b_j, c_j]$ 。

每条优惠券线段最多分配给一个纪念品。

如果分配给某个纪念品，它可以覆盖该纪念品线段上的对应部分。

求最大总覆盖长度。

最后答案就是：

$$\sum_i a_i - \text{最大总覆盖长度}$$

第四个观察：只需要按价格层扫描

考虑价格轴上的某个位置 x 。

有多少个纪念品需要支付这一层？

$$S(x) = |\{i \mid a_i > x\}|$$

有多少张优惠券能覆盖这一层？

$$C(x) = |\{j \mid b_j \leq x < c_j\}|$$

在这一层，最多能抵消：

$$\min(S(x), C(x))$$

因为：

- 纪念品需求只有 $S(x)$ 份；
- 优惠券供给只有 $C(x)$ 份；

- 每张优惠券在同一价格层最多服务一个纪念品。

关键是：对于区间问题，这个上界可以达到。

可以从高到低扫描价格轴：

- 纪念品 $[0, a_i]$ 在 a_i 处开始出现；
- 优惠券 $[b_j, c_j]$ 在 c_j 处开始有效，在 b_j 处结束；
- 每当有空的纪念品轨道和可用优惠券，就把优惠券放上去。

由于所有纪念品区间都从 0 开始，优惠券区间也是连续区间，所以这种逐层匹配不会产生额外冲突。

因此最大总覆盖长度就是：

$$\int \min(S(x), C(x)) dx$$

由于所有端点都是整数，我们只需要在所有端点之间做扫描线。

最终算法

令：

$$\text{base} = \sum_i a_i$$

对于每张有效优惠券 $c_j > b_j$ ，加入区间：

$$[b_j, c_j)$$

对于每个纪念品，加入区间：

$$[0, a_i)$$

扫描所有端点。

在相邻两个端点 $[l, r)$ 之间：

- 当前仍未结束的纪念品数量为 s ；
- 当前有效优惠券数量为 k 。

这一段的最大覆盖贡献为：

$$(r - l) \cdot \min(s, k)$$

最后：

$$\text{answer} = \sum_i a_i - \sum_{\text{segments}} (r - l) \min(s, k)$$

复杂度：

$$O((n + m) \log(n + m))$$

3. Trick 与陷阱

Trick 点拨

这题的核心 Trick 是：

把“优惠券影响下一次购买”转化成价格轴上的区间覆盖。

优惠券 (b, c) 不是一个普通折扣，而是覆盖价格区间：

$$[b, c]$$

纪念品价格 a 是需要支付的区间：

$$[0, a]$$

于是购买顺序优化消失了，变成了扫描线统计：

$$\min(\text{当前纪念品数量}, \text{当前优惠券数量})$$

易错点

1. $c_j \leq b_j$ 的优惠券可以直接忽略。

它对应的区间长度非正，不可能带来收益。

2. 答案必须用 `long long`。

最大花费量级是：

$$5 \cdot 10^5 \cdot 10^9 = 5 \cdot 10^{14}$$

3. 端点要按半开区间处理。

使用：

$$[0, a_i), [b_j, c_j)$$

端点本身是否包含不影响长度。

4. 同一坐标的所有事件要一起处理。

先计算上一段 $[last, x)$ 的贡献，再统一更新当前坐标的事件。

5. 价格为 0 的纪念品和 $b_j = 0$ 的优惠券要正确处理。

4. 代码实现 C++23, jiangly Style

```
#include <bits/stdc++.h>

using namespace std;

using i64 = long long;

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
```

```

int T;
cin >> T;

while (T--> 0) {
    int n, m;
    cin >> n >> m;

    vector<i64> a(n), b(m), c(m);
    vector<array<i64, 3>> e;

    e.reserve(n + 2 * m);

    i64 ans = 0;

    for (auto &x : a) {
        cin >> x;
        ans += x;
        e.push_back({x, -1, 0});
    }

    for (auto &x : b) {
        cin >> x;
    }

    for (auto &x : c) {
        cin >> x;
    }

    for (int i = 0; i < m; i++) {
        if (c[i] > b[i]) {
            e.push_back({b[i], 0, 1});
            e.push_back({c[i], 0, -1});
        }
    }

    sort(e.begin(), e.end());

    i64 s = n;
    i64 k = 0;
    i64 last = 0;
    i64 cover = 0;

    for (int i = 0; i < int(e.size()); ) {
        int j = i;
        i64 x = e[i][0];

        cover += (x - last) * min(s, k);

        while (j < int(e.size()) && e[j][0] == x) {
            s += e[j][1];
            k += e[j][2];
            j++;
        }
    }
}

```

```
        }

        last = x;
        i = j;
    }

    cout << ans - cover << '\n';
}

return 0;
}
```

5. 总结与启示

难度评分

预估 Codeforces Rating: **2400 ~ 2600**。

代码很短，但思维转化非常关键。

核心启示

当“顺序决策”难以处理时，尝试把每个操作解释成对某个连续区间的贡献。

同类习题推荐

1. Codeforces 1000C - Covered Points Count

经典扫描线统计区间覆盖次数。

2. AtCoder ABC188 D - Snuke Prime

扫描线 + 每一段取上限，和本题的 $\min(S, C)$ 思路很像。

3. Codeforces 1249D2 - Too Many Segments

区间覆盖容量限制下的贪心与扫描线。